

Q1)

NoSQL means “**Not Only SQL.**” It refers to databases that store data in flexible ways and are useful for large-scale or fast-changing data. The four main NoSQL families are:

1. Key-Value - Redis - storing user sessions/cache data
2. Document -MongoDB -storing customer/application documents
3. Column-Family-Cassandra-storing large transaction or activity data
4. Graph - Neo4j-finding relationships between customers, accounts and transactions

Q2)

CAP theorem: In a distributed system, when a network partition happens, you cannot guarantee both Consistency and Availability at the same time.

“Choose two of three” is misleading because you don't simply choose two permanently. Partition tolerance is generally necessary in distributed systems, and the real choice during a network partition is mainly between Consistency (C) and Availability (A).

System	Gives up	Why
Core banking ledger	Availability	It is better to temporarily reject/delay a transaction than show or create an incorrect account balance.
Balance cache	Consistency	A slightly old balance in cache is acceptable because the correct balance can be obtained from the main database.

Session store	Consistency	If a session update is temporarily delayed, the user can log in again or the session can be recreated; keeping the service available is more useful.
---------------	-------------	--

Q3) Data can be lost if Redis fails

If Redis crashes before the data is safely persisted, the latest balance update could be lost.

Failure: A customer's account may show an old balance.

Redis is mainly an in-memory system

Redis keeps data primarily in RAM, so it should not be treated like the permanent banking ledger.

Failure: A restart or serious failure could make important balance data unavailable or lost.

Banking balances need strong and reliable transactions

Account balance changes must be permanently recorded and correctly controlled.

Redis is better suited for **cache/session/fast-access data** than as the authoritative financial record.

Failure: A debit or credit could be recorded incorrectly, causing an incorrect account balance.

Q4)

Redis executes commands one at a time.

Advantage:

Commands are processed sequentially, so individual operations are atomic and easy to reason about.

Banking example:

If two customers simultaneously update a transaction counter, Redis processes the INCR operations one by one, helping prevent conflicting updates.

Danger: A slow or expensive command can block other commands because other requests have to wait.

Banking example: If a very large Redis operation takes a long time, other users trying to access cached account information may have to wait, increasing response time.

Q5)

Requirement	Redis Data Structure	Commands
1. Last ten transactions, newest first, never more than ten	List	LPUSH, LTRIM, LRANGE
2. Check whether a card is blocked in constant time	Set	SADD, SISMEMBER, SREM
3. Three highest balances in a branch, without sorting	Sorted Set (ZSET)	ZADD, ZREVRANGE
4. Count balance endpoint calls today	String (counter)	INCR, GET, EXPIRE
5. Customer session, update one field without rewriting rest	Hash	HSET, HGET, HGETALL
6. Maximum five OTP requests per customer per hour, sliding window	Sorted Set (ZSET)	ZADD, ZREMRANGEBYSCORE, ZCARD
7. Nearest ATM within 2 km	Geospatial	GEOADD, GEOSEARCH

Q6)

Cache-aside: The application reads from the cache first. If the data is missing, it reads from the database and then puts the data into Redis. On a write, the application normally updates the database and invalidates/updates the cache. The first read after a write may get a cache miss and load the latest value from the database.

Write-through: Every write updates the cache and database together. The cache is populated during the write, so the first read after the write gets the newly written value directly from the cache. The write costs more because both cache and database must be updated.

Q7)

Write-behind is unsafe for account balances because the cache is updated first and the database update happens later. If the cache update succeeds but the system crashes before the database is updated, the balance in the database can remain old and the money-related transaction can be lost.

Example sequence:

1. Account balance in DB = ₹10,000.
2. Customer deposits ₹5,000.
3. Redis is updated to ₹15,000.
4. Before Redis writes ₹15,000 to the database, the system crashes.
5. Database still says ₹10,000 → the ₹5,000 update is lost.

Write-behind can be legitimate for workloads such as **non-critical analytics, page-view counters, or temporary statistics**, where losing a small number of updates is acceptable.

Q8)

1. Delete rather than overwrite:

After changing data in the database, deleting the cached value is safer than trying

to overwrite it. This prevents the cache from accidentally containing an incorrect or partially updated value.

2. Delete after commit:

The cache should be deleted after the database transaction commits. If we delete it before the transaction commits and the transaction later rolls back, the next read may load the old value into the cache, creating an inconsistent result.

Q9)

TTL is a backup safety mechanism for invalidation.

Even when invalidation has been implemented correctly, bugs, missed code paths, application failures, or unusual situations can leave stale data in Redis. A TTL ensures that the cached value eventually expires, limiting how long stale data can remain in the cache.

Simple idea:

Invalidation removes stale data normally; TTL is the safety net if invalidation fails.

Q10)

A cache stampede happens when a popular cache entry expires, and many requests arrive at the same time. Since the data is missing, all requests go to the database simultaneously, causing a sudden heavy load.

Three ways to prevent it:

1. Lock/mutex – allow only one request to fetch the data from the database; others wait.
2. Early refresh – refresh the cache before it actually expires.
3. Randomized TTL/jitter – give similar cache entries slightly different expiration times so they do not expire together.

I would implement a lock/mutex first because it directly prevents multiple requests from fetching the same missing data from the database at the same time.

Q11)

If maxmemory is unset in production, Redis can keep consuming memory as more data is added. Eventually, the server can run out of available RAM, causing OS-level memory pressure or an OOM kill, which can affect Redis and potentially other processes.

This is worse than having a controlled eviction policy because there is no predictable limit on Redis memory usage.

Trap with volatile-lru:

volatile-lru removes the least recently used keys only from keys that have a TTL. If most of the Redis keys do not have an expiration time, Redis may have few or no eligible keys to evict. Therefore, memory can still grow and the expected eviction may not happen.

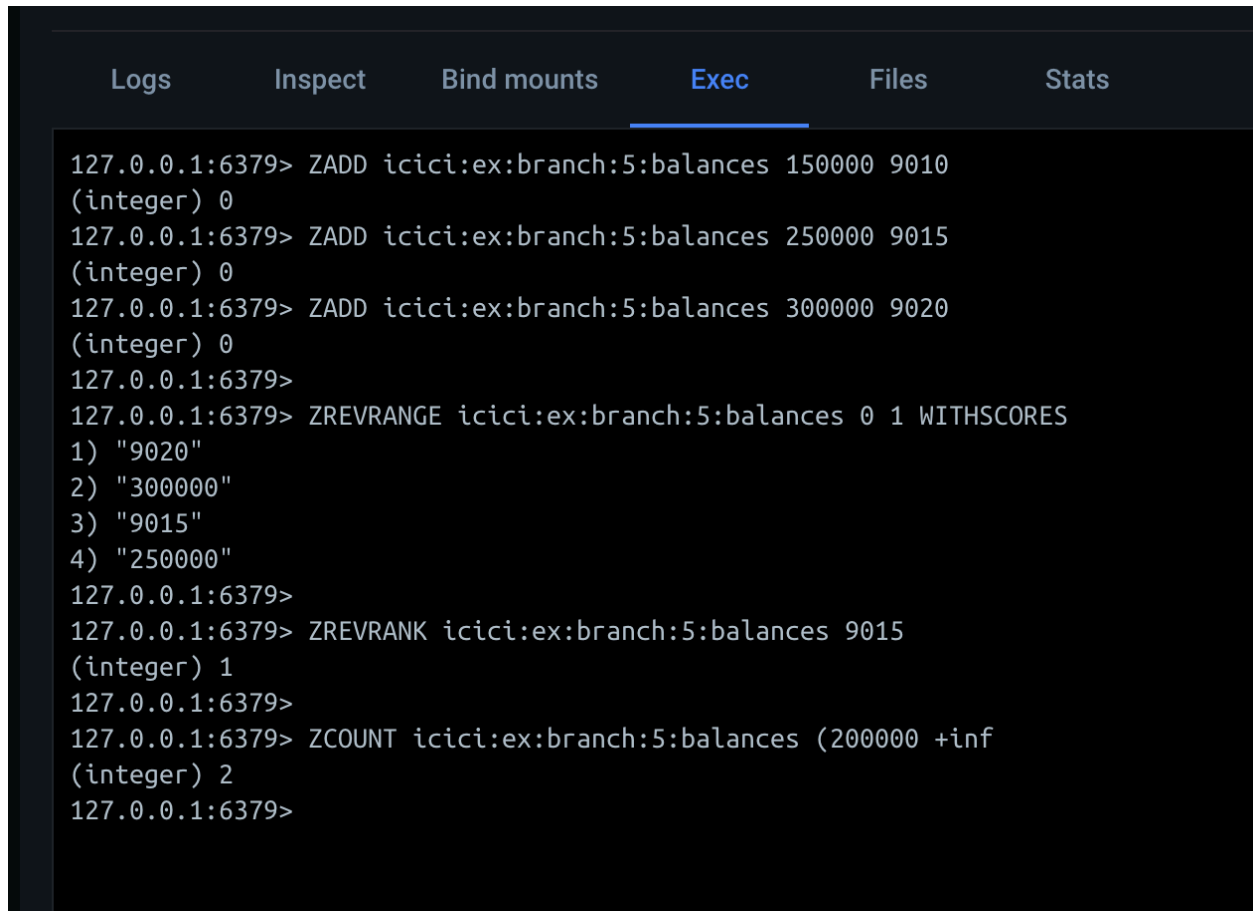
Q12)

Possible cause	How to identify it
1. TTL is too short	Check key TTLs and compare cache misses with expiration times.
2. Keys are being deleted/invalidated too often	Monitor invalidation/delete operations and compare them with misses.
3. Poor key design	Examine cache keys and see whether the same account is being stored under different keys.
4. Cache is too small / eviction is happening	Check INFO stats and eviction metrics such as evicted_keys.

AND:

```
127.0.0.1:6379> LPUSH icici:ex:account:9010:transactions "TXN1"
(integer) 6
127.0.0.1:6379> LPUSH icici:ex:account:9010:transactions "TXN2"
(integer) 7
127.0.0.1:6379> LPUSH icici:ex:account:9010:transactions "TXN3"
(integer) 8
127.0.0.1:6379> LPUSH icici:ex:account:9010:transactions "TXN4"
(integer) 9
127.0.0.1:6379> LPUSH icici:ex:account:9010:transactions "TXN5"
(integer) 10
127.0.0.1:6379> LTRIM icici:ex:account:9010:transactions 0 4
OK
127.0.0.1:6379>
127.0.0.1:6379> LPUSH icici:ex:account:9010:transactions "TXN6"
(integer) 6
127.0.0.1:6379> LTRIM icici:ex:account:9010:transactions 0 4
OK
127.0.0.1:6379>
127.0.0.1:6379> LRANGE icici:ex:account:9010:transactions 0 4
1) "TXN6"
2) "TXN5"
3) "TXN4"
4) "TXN3"
5) "TXN2"
127.0.0.1:6379> █
```

ANS:



```
127.0.0.1:6379> ZADD icici:ex:branch:5:balances 150000 9010
(integer) 0
127.0.0.1:6379> ZADD icici:ex:branch:5:balances 250000 9015
(integer) 0
127.0.0.1:6379> ZADD icici:ex:branch:5:balances 300000 9020
(integer) 0
127.0.0.1:6379>
127.0.0.1:6379> ZREVRANGE icici:ex:branch:5:balances 0 1 WITHSCORES
1) "9020"
2) "300000"
3) "9015"
4) "250000"
127.0.0.1:6379>
127.0.0.1:6379> ZREVRANK icici:ex:branch:5:balances 9015
(integer) 1
127.0.0.1:6379>
127.0.0.1:6379> ZCOUNT icici:ex:branch:5:balances (200000 +inf
(integer) 2
127.0.0.1:6379>
```

ANS:

Logs	Inspect	Bind mounts	Exec	Files	Stats
------	---------	-------------	------	-------	-------

```
127.0.0.1:6379> LPUSH icici:ex:rm:101:recent 9010
(integer) 1
127.0.0.1:6379> LPUSH icici:ex:rm:101:recent 9015
(integer) 2
127.0.0.1:6379> LPUSH icici:ex:rm:101:recent 9020
(integer) 3
127.0.0.1:6379> LPUSH icici:ex:rm:101:recent 9030
(integer) 4
127.0.0.1:6379> LPUSH icici:ex:rm:101:recent 9040
(integer) 5
127.0.0.1:6379> LRANGE icici:ex:rm:101:recent 0 4
1) "9040"
2) "9030"
3) "9020"
4) "9015"
5) "9010"
127.0.0.1:6379> LREM icici:ex:rm:101:recent 0 9015
(integer) 1
127.0.0.1:6379> LPUSH icici:ex:rm:101:recent 9015
(integer) 5
127.0.0.1:6379> LTRIM icici:ex:rm:101:recent 0 4
OK
127.0.0.1:6379> LRANGE icici:ex:rm:101:recent 0 4
1) "9015"
2) "9040"
3) "9030"
4) "9020"
5) "9010"
127.0.0.1:6379>
```

16.ANS:

```
async function getCustomerProfile(db, customerId) {
  const key = `icici:ex:customer:${customerId}:profile`;

  const cached = await redis.hGetAll(key);

  if (Object.keys(cached).length > 0) {
    console.log("CACHE HIT");
    return cached;
  }

  console.log("CACHE MISS");

  const result = await db.query(`
    SELECT first_name, last_name, city, kyc_status, risk_rating
    FROM customer
    WHERE customer_id = $1
  `, [customerId]);

  if (result.rows.length === 0) {
    return null;
  }

  const customer = result.rows[0];

  await redis.hSet(key, {
    first_name: customer.first_name,
    last_name: customer.last_name,
    city: customer.city,
    kyc_status: customer.kyc_status,
    risk_rating: customer.risk_rating
  });

  await redis.expire(key, 3600);

  return customer;
}
```

17.

ANS:

```
async function getBranchSummary(db, branchId) {
  const key = `icici:ex:branch:${branchId}:summary`;

  const cached = await redis.hGetAll(key);

  if (Object.keys(cached).length > 0) {
    console.log("CACHE HIT");
    return cached;
  }

  console.log("CACHE MISS");

  const result = await db.query(`
    SELECT
      COUNT(DISTINCT c.customer_id) AS customer_count,
      COUNT(DISTINCT a.account_number) AS account_count,
      COALESCE(SUM(a.balance), 0) AS total_deposits
    FROM account a
    JOIN customer c ON a.customer_id = c.customer_id
    WHERE a.branch_id = $1
  `, [branchId]);

  const summary = result.rows[0];

  await redis.hSet(key, summary);
  await redis.expire(key, 600);

  return summary;
}
```

18.

ANS:

```
async function invalidateCustomer(db, customerId) {
  const result = await db.query(`
    SELECT account_number
    FROM account
    WHERE customer_id = $1
  `, [customerId]);

  const keys = [
    `icici:ex:customer:${customerId}:profile`
  ];

  for (const row of result.rows) {
    keys.push(`icici:ex:account:${row.account_number}:profile`);
  }

  if (keys.length > 0) {
    await redis.del(keys);
  }
}
```

19.ANS:

```
const branches = [1,2,3,4,5,6,7,8];

async function benchmark(db, useCache) {
  let dbCalls = 0, hits = 0;
  const start = Date.now();

  for (let i = 0; i < 500; i++) {
    const id = branches[i % 8];
    const key = `icici:ex:branch:${id}:summary`;

    if (useCache && Object.keys(await redis.hGetAll(key)).length) {
      hits++;
      continue;
    }

    await db.query(`SELECT COUNT(*) FROM account WHERE branch_id=$1`, [id]);
    dbCalls++;
  }

  console.log("Time:", Date.now() - start, "ms");
  console.log("DB calls:", dbCalls);
  console.log("Hit rate:", (hits / 500 * 100).toFixed(2) + "%");
}

await benchmark(db, true);
await benchmark(db, false);
```

20.

ANS:

```
async function testStaleCache(db, customerId) {
  const key = `icici:ex:customer:${customerId}:profile`;

  // Cache profile
  await getCustomerProfile(db, customerId);

  // Change database directly
  await db.query(
    `UPDATE customer SET city = 'Delhi' WHERE customer_id = $1`,
    [customerId]
  );

  // Shows stale cached value
  console.log("Before fix:", await getCustomerProfile(db, customerId));

  // Fix: delete cache
  await redis.del(key);

  // Gets fresh value from database
  console.log("After fix:", await getCustomerProfile(db, customerId));
}
```

21.

ANS:

```
async function cachePerformance(db, branchId) {
  const key = `icici:ex:branch:${branchId}:performance`;
  const start = Date.now();

  let data = await redis.get(key);

  if (!data) {
    const result = await db.query(`
      SELECT *
      FROM branch_performance
      WHERE branch_id = $1
    `, [branchId]);

    data = result.rows;

    const ttl = 900 + Math.floor(Math.random() * 60);

    await redis.set(key, JSON.stringify(data), { EX: ttl });
  }

  const time = Date.now() - start;
  console.log("Time:", time, "ms");

  return data;
}
```

22.

ANS:

```
async function createSession(sessionId, customerId, ipAddress) {
  const key = `icici:ex:session:${sessionId}`;

  await redis.hSet(key, {
    customer_id: customerId,
    ip_address: ipAddress,
    last_seen: new Date().toISOString()
  });

  await redis.expire(key, 1800);
}

async function active_sessions(customerId, sessionKeys) {
  const sessions = [];

  for (const key of sessionKeys) {
    const session = await redis.hGetAll(key);

    if (session.customer_id == customerId) {
      sessions.push({
        session_id: key,
        ip_address: session.ip_address,
        last_seen: session.last_seen,
        remaining_ttl: await redis.ttl(key)
      });
    }
  }

  return sessions;
}
```

23.

ANS:

```
async function failedLogin(accountId) {
  const key = `icici:ex:loginfail:${accountId}`;
  const lockKey = `icici:ex:loginlock:${accountId}`;

  if (await redis.exists(lockKey)) {
    return "ACCOUNT LOCKED";
  }

  const count = await redis.incr(key);

  if (count === 1) {
    await redis.expire(key, 900);
  }

  if (count >= 5) {
    await redis.set(lockKey, "1", { EX: 1800 });
    return "ACCOUNT LOCKED";
  }

  return `Failed attempt ${count}`;
}
```

24.

ANS:

```
async function is_card_blocked(cardId) {
  return await redis.sIsMember(
    "icici:ex:blocked-cards",
    cardId
  );
}

async function loadBlockedCards(db) {
  const result = await db.query(
    "SELECT card_id FROM blocked_cards"
  );

  for (const row of result.rows) {
    await redis.sAdd(
      "icici:ex:blocked-cards",
      row.card_id
    );
  }
}

async function blockCard(cardId) {
  await redis.sAdd(
    "icici:ex:blocked-cards",
    cardId
  );
}
```

Q25)

1. Customer's account balance: Yes, cache it for a short time (e.g., 30–60 seconds). Balance changes frequently, so a long TTL can show an incorrect balance.
2. Result of a fund transfer: Do not cache as the source of truth. A transfer is a financial transaction and must be confirmed from the database/transaction system.
- 3 Bank's branches and IFSC codes: Yes, cache it for 1 day or longer, because this information changes rarely.
4. Today's USD to INR exchange rate: Yes, cache it for a short period, such as 5–15 minutes, because rates can change during the day.
5. Customer's transaction history for seven years: Do not cache the complete history. It is large and rarely requested. Fetch it from the database, or cache only frequently requested recent records.
6. Whether a card is blocked: Yes, but use a very short TTL or update it immediately when the card status changes, because showing an old status can create a security problem.
7. One-time password (OTP): Yes, cache it for 2–5 minutes, matching the OTP's expiry time.
8. Number of failed login attempts: Yes, keep it in Redis with a short expiry, because it is useful for rate limiting/security.

Q26)

I would first ask the mobile team how often balances change and how much stale data the business can tolerate. I would also ask whether customers can make withdrawals, deposits, UPI payments or transfers during those 24 hours. I would not use a 24-hour TTL for account balances. I would use a much shorter time, such as 30–60 seconds, and update/remove the cached balance whenever a successful transaction changes the balance.

The 99.9% hit rate is not the main goal. Correctness is more important than a very high cache hit rate for bank balances. We should decide the number by measuring how often balances change, the acceptable delay, database load and customer experience.

Q27)

Balance endpoint: It may not be able to read the cached balance. It should fall back to the database. This is acceptable because the database is the source of truth. I would add proper timeout and fallback handling.

Session store: Users may be logged out or unable to continue their sessions if sessions exist only in Redis. This is not ideal. I would use a highly available Redis setup and provide a safe session-recovery approach.

Rate limiter: The rate limiter may stop working or allow requests without proper limits. This can be a security problem. I would use Redis with high availability and a safe fallback limit when Redis is unavailable

Q28)

I would not agree with this design. Redis is a fast data store/cache, but the customer's balance should be permanently stored in the database as the source of truth. If Redis fails before the nightly database update, we could lose hours of transactions and balance changes. We could also lose deposits, withdrawals, transfers and their transaction history. A nightly update also means the database may contain an old balance for many hours. For a banking system, this can cause incorrect balances and financial reconciliation problems. Redis should improve database performance, not replace the permanent transaction database.

Q29)

Three examples are:

1. ATM cash withdrawal
2. UPI/payment transaction

3. Bank employee or back-office adjustment

These can change the database balance without the application's `transfer()` function updating the cache. The cache can then contain an old balance. A better architecture is to have all balance changes go through a common transaction/service layer. After a successful database transaction, it should publish an event such as `BalanceChanged`. A cache-update service can then update or remove the Redis value.

The cost is more complexity, infrastructure and monitoring. It also requires reliable event processing and handling failed events.

Q30)

I would not automatically preload every account just to remove the remaining 4% misses. Pre-loading millions of accounts would use a large amount of Redis memory, increase startup time and put extra load on the database while loading the data. Some accounts may never even be accessed.

A 96% hit rate is already good if the remaining database requests are acceptable. We should first check why the 4% are misses and whether they actually cause a database-load problem. Pre-loading can be useful when the number of accounts is manageable and almost every account is frequently accessed, or when very fast access is required for a known set of accounts.

Q31)

The balance caching reduced the database load by the measured amount from our testing. Instead of asking the database for the same balance every time, we keep a temporary copy for quick access. The original balance remains safely stored in the database as the main source of truth. If the temporary copy is slightly old, the customer may briefly see an older balance. The main risk is that a balance-changing transaction may happen while the copy is still old.

To reduce this risk, the copy is kept only for a short period and is refreshed after balance changes. The database is always used when the temporary copy is missing or cannot be trusted. This limits how long an incorrect value can be shown. We also tested the system behaviour when the Redis service is unavailable. For the internet banking session problem, the system now has proper handling when the session store is unavailable. Users are not allowed to continue with an invalid or missing session.

The changes improve performance while keeping the database as the final source of truth.